

RP-05 — Durable Storage Solution Evaluation: Group Lesson Planner

Metadata

Field	Value
Report id	RP-05
Date	2026-08-20
Subject	Durable persistence for a static, backendless, single-user app on GitHub Pages
Inputs consumed	RP-01 §3 data model and §4 storage map (taken as given, not re-derived); the research agent contract; <code>index.html</code> in the research worktree, read at lines 359-408 and 1186-1213
Runtime-verified by me	Storage capability and quota on the live origin <code>https://denyslystopadskyy.github.io</code> , measured in Chromium 151 headless via <code>navigator.storage.estimate()</code> and <code>navigator.storage.persisted()</code> ; File System Access API presence; secure-context status. MDN's Baseline status for <code>showSaveFilePicker()</code> read from the rendered page. Firebase pricing figures read from the rendered (JavaScript-built) pricing table
Statically read	<code>index.html:377-378</code> (two config storage keys), <code>index.html:382-400</code> (default template block), <code>index.html:1188-1213</code> (whole storage service)
Externally sourced	All quotas, prices, scope classifications, token lifetimes, eviction policies and inactivity policies come from the vendor or standards documentation listed in §9, fetched 2026-08-20
Not verified	Nothing in this report was tested against a live Firebase, Supabase, Cloudflare, Drive or Dropbox account. All provider behaviour is documentation-sourced and labelled as such

Executive summary

The data was not lost to an exotic browser policy. It was lost because **the only copy lived on one device**, and every browser-side storage type dies together when a profile is wiped. `localStorage`, `sessionStorage`, `IndexedDB`, the Cache API and OPFS are all destroyed by profile cleanup, by "clear cookies and other site data" [S4], and by device replacement. No client-side storage choice fixes this. Only an off-device copy does. Everything else in §1 — quota-pressure eviction, WebKit's seven-day cap on script-writable storage [S1][S2] — is a *future* risk to a local-only store, not the cause of what happened.

Three verified facts decide the recommendation:

1. **Supabase is disqualified as the sole store.** Supabase's own production checklist states it may pause Free Plan applications that show low activity in a seven-day period, and that Free Plan database backups are not available for download [S11]. A teacher's quiet July is longer than seven days. That is the original incident with a new mechanism.

2. **A browser-only Google Drive client cannot hold a refresh token.** Google's supported in-browser token model issues no refresh token and requires a **user-driven event** such as a button press to obtain each new short-lived access token [S22]. Worse, while the OAuth app sits in *Testing* publishing status, "Authorizations by a test user will expire seven days from the time of consent" [S26] and refresh tokens for Testing-status apps expire in seven days [S19]. Drive `appDataFolder` remains a credible option — the scope is classified **non-sensitive**, so no security assessment or restricted-scope review applies [S21] — but it costs the teacher a click every session and hides the file from her.
3. **Cloud Firestore on the Spark plan needs no payment method** [S6], documents that its client API key is not a secret and is safe in checked-in code [S9], keeps the user signed in across browser restarts by default [S10], can be pinned to `europe-central2` in Warsaw, Poland [S8], and has no documented inactivity-pause policy at its quota or pricing pages [S6][S7]. At this app's scale — RP-01 measured **354 bytes** for one group with five dates over two months — one Firestore document per user consumes roughly 0.001% of the 1 GiB free allowance and roughly 1.4% of the 1 MiB per-document limit [S7].

Recommendation. Primary: **one Cloud Firestore document per signed-in user, written behind an unchanged local cache**, with Google sign-in as the only login. Fallback, and also a prerequisite that ships first: **hardened local storage plus a versioned one-click JSON backup with a visible staleness indicator**. Runner-up hosted option, with its selecting conditions stated in §6: Google Drive `appDataFolder` .

Rejected as disproportionate (§5): Postgres in any form, CRDT sync engines, and edge databases. This app stores about 14 KB and is written a few dozen times a month by one person. Yjs, Automerge, ElectricSQL and RxDB replication all exist to reconcile concurrent writers; there is exactly one writer, so the correct conflict policy is last-write-wins on a timestamp, and every one of those engines still needs a server that does not exist [S34][S35].

One ordering constraint that overrides everything. The default payment template hardcodes a real person's full name, bank IBAN and tax identification number (`index.html:387–392` , plus a personal first name at `index.html:400`). Adding *any* cloud sync before that is fixed replicates a live financial identifier into a third-party datastore and into a provider's backups. RP-01's first quick win must land before §7 is implemented. This is stated again in §8 as a blocker.

1. Storage loss root-cause analysis

Mechanism	Storage types affected	Documented behaviour	Applies here
Laptop or OS profile cleanup	All origin-scoped storage: Web Storage, IndexedDB, Cache API, OPFS, service workers, cookies	No web specification protects data against deletion of the browser profile directory. Every storage type in the origin's bucket is removed together [S2]	Y — this is the reported cause

Mechanism	Storage types affected	Documented behaviour	Applies here
Chrome "Clear browsing data", Cookies and other site data	Web Storage, IndexedDB, Web SQL, application caches	Chrome deletes "HTML5-enabled storage types including application caches, Web Storage data, Web SQL Database data, and Indexed Database data" [S4]	Y as a future repeat vector
Private or incognito session	<code>localStorage</code> written in that session	"localStorage data for a document loaded in a private browsing or incognito session is cleared when the last private tab is closed" [S3]	Y if she ever works in a private window
Quota-pressure eviction, best-effort mode	The whole origin bucket, deleted atomically	Under device storage pressure the browser evicts least-recently-used origins; all of an origin's data goes at once so partial state cannot result. Origins granted persistence are skipped [S2]	N at 14 KB, unless the disk itself is nearly full
Browser-wide storage cap exceeded	The whole origin bucket	When all origins together exceed the browser's overall limit (Chromium: 80% of disk for browser apps) best-effort origins are evicted LRU [S2]	N at this volume
WebKit seven-day cap on script-writable storage	IndexedDB, <code>localStorage</code> , <code>sessionStorage</code> , media keys, Service Worker registrations and cache	WebKit deletes "all of a website's script-writable storage after seven days of Safari use without user interaction on the site". Home-screen web apps keep their own counter and are not expected to be cleared [S1]. MDN describes the same policy as eviction of an origin's script-created data after seven days without a click or tap, applied when cross-site tracking prevention is on and the origin has not been granted persistence [S2]	TBD — Y only if Safari or an iPad is in use. Resolved by asking which browser and device she uses
Web Storage hard cap, not eviction	<code>localStorage</code> only	Web Storage is capped separately from the quota-managed bucket at a maximum of 10 MiB per origin, split 5 MiB <code>localStorage</code> plus 5 MiB <code>sessionStorage</code> , and throws <code>QuotaExceededError</code> beyond it [S2]	N at 14 KB, but the throw is completely unguarded — see D14 in RP-01
App-level corrupt-value abort	<code>groupLessonPlannerData</code>	Not a browser mechanism. <code>JSON.parse</code> runs unguarded at <code>index.html:1192</code> ; RP-01 runtime-verified that a truncated value aborts <code>App.init()</code> and leaves a dead page with no recovery route	Y — the most likely mechanism for the next loss
App's own destructive paths	All three keys	<code>clear()</code> at <code>index.html:1203-1206</code> removes two keys; CSV import replaces all data with no confirmation (RP-01 D11); a balanced stray quote in a CSV silently destroys existing data (RP-01 D12)	Y — user-triggerable data loss with no undo

Mechanism	Storage types affected	Documented behaviour	Applies here
Device replacement	All client-side storage	Nothing migrates automatically between machines	Y

What actually happened, and what would be immune

The incident description is "the laptop was cleaned". Whatever the operator meant — a wipe and reinstall, a new profile, or a privacy-cleaner sweep — the outcome is the same class of event: the browser profile that held the three `localStorage` keys ceased to exist. This deserves stating plainly because it changes the shape of the answer: **no choice of client-side storage API would have helped**. Migrating `localStorage` to IndexedDB, to OPFS, or to the File System Access API buys durability against *quota* pressure and nothing against profile deletion. Only a copy that is not on that device is immune, and of the candidate families exactly three produce one: hosted backend-as-a-service, user-owned cloud file storage, and a file the user deliberately exports and keeps somewhere else.

`navigator.storage.persist()` — worth doing, insufficient on its own

`navigator.storage.persist()` moves an origin from best-effort to persistent mode. In persistent mode the browser evicts the origin's data only if the user deletes it via browser settings [S2]. Chromium, Edge and Safari auto-approve or auto-deny based on the user's engagement history without prompting; Firefox shows a permission popup [S2]. Granting persistence also exempts the origin from WebKit's seven-day script-storage eviction, since that policy applies only to origins not granted persistence [S2].

Measured on the live origin in Chromium 151 headless: `navigator.storage.persisted()` returned `false`, and `navigator.storage.persist` is present. So the app is currently in best-effort mode and has never asked. Requesting persistence is a two-line change with real value against eviction and against the WebKit cap — and **zero** value against the profile wipe that actually occurred. That is why it appears in §10 as a quick win and nowhere in §6 as a solution.

Quota measured, and a correction to the scope of RP-01's open item

RP-01 left "localStorage quota for this origin" as TBD. This report partly resolves it and partly sharpens it. Runtime-verified on <https://denyslystopadskyy.github.io> in Chromium 151 headless:

`navigator.storage.estimate()` returned `quota = 10,737,418,240` bytes (10 GiB) and `usage = 0`, in a secure context. That figure describes the **quota-managed bucket** — IndexedDB, Cache API, OPFS — and does *not* describe `localStorage`, which MDN documents as separately capped at 5 MiB per origin [S2]. The app stores everything in `localStorage`, so the 10 GiB number is not its ceiling; the documented 5 MiB is, and against a measured 354-byte dataset that is a non-issue either way. The exact `localStorage` cap for this browser was not probed because the available browser tool blocks Web Storage access; a manual probe in the teacher's own browser would settle it, and it does not matter.

Also verified present in that browser: `window.showSaveFilePicker`, `window.showOpenFilePicker`, and `navigator.storage.getDirectory` (OPFS). MDN marks `showSaveFilePicker()` as "Limited availability — This feature is not Baseline because it does not work in some of the most widely-used browsers" [S5], so it can only be a Chromium-only enhancement, never the mechanism a backup depends on.

2. Candidate long-list with exclusion reasons

Candidate	Family	Verdict	Reason
Cloud Firestore, Firebase Spark plan	BaaS	Shortlisted	Free tier needs no payment method [S6]; client key documented as non-secret [S9]; EU region in Warsaw [S8]; no documented inactivity pause
Supabase Postgres, Free Plan	BaaS	Shortlisted, then rejected	Verified pause of low-activity Free Plan apps in a 7-day period, and no downloadable Free Plan backups [S11]. See §4
Cloudflare Worker plus Workers KV	Edge	Shortlisted, then rejected	Limits are ample [S16][S17], but a Worker has no user identity, so a static client must present a shared secret that is published in page source. Adding an identity layer is maintainer work with no user benefit here
Cloudflare D1	Edge	Excluded	Same credential problem as KV, plus a relational schema for a 14 KB document. 500 MB max database, 5 GB per account on the free plan [S18] — the limits are irrelevant; the shape is wrong
Google Drive appDataFolder	User-owned file	Shortlisted, runner-up	<code>drive.appdata</code> is a non-sensitive scope [S21], so no verification gauntlet. Costs one click per session [S22] and the file is hidden from the user [S20]
Google Drive user-visible file via <code>drive.file</code> plus Picker	User-owned file	Excluded	<code>drive.file</code> is also classified non-sensitive [S21], and a visible file would fix the one real weakness of <code>appDataFolder</code> — that the folder is hidden from her [S20], making "she owns her data" nominal. It loses anyway: it inherits the same no-refresh-token constraint [S22] and adds the Google Picker to the flow, so it costs her strictly more clicks than the option it improves on
Dropbox App folder	User-owned file	Shortlisted, then rejected	PKCE for public clients is supported [S27] and development status allows up to 500 linked users [S28], so it is technically viable — but the teacher would need a Dropbox account she probably does not have. Strictly worse than Drive on operability
Microsoft OneDrive app folder	User-owned file	Excluded	Same shape as Drive and Dropbox with no advantage for this user, and a third account to create. Excluded on operability, not capability. Limits and token policy not researched

Candidate	Family	Verdict	Reason
GitHub Gist API	Git-backed	Excluded	Gist creation and update require authentication and the <code>gist</code> scope [S29]. A token shipped in client JavaScript on a public GitHub Pages site is a published write credential; GitHub secret scanning runs automatically on public repositories and reports partner secrets to the provider [S30], so the credential is also liable to be revoked, breaking sync silently
Private repo via GitHub Contents API	Git-backed	Excluded	Same published-credential problem, with broader blast radius: the same token can rewrite the deployed app
Vercel or Netlify function plus hosted Postgres	Serverless	Excluded	Introduces a build pipeline, a serverless project and a database to run one 14 KB write. Neon's free plan forces scale-to-zero after 5 minutes and cannot disable it [S31]; the pipeline is three moving parts where zero exist today
Appwrite Cloud	BaaS	Excluded	Free plan caps at 1 organisation, 2 projects, 1 database per project, and its free-plan page documents no inactivity policy at all [S32] — an unknown durability posture is worse than a known one for the single requirement that matters
Nhost	BaaS	Excluded	Not researched to primary sources. Excluded because Firestore already satisfies every constraint and adding a fourth BaaS to the comparison would add no decision value. Inactivity policy TBD
PocketBase	Self-hosted BaaS	Excluded	Deployed by "uploading the executable on your server" [S33]. There is no server and no budget for one. A VM the maintainer forgets to renew is a worse durability story than the browser
RxDB replication	Local-first sync	Excluded	The replication protocol requires a backend implementing <code>pullHandler</code> , <code>pushHandler</code> and <code>pullStream</code> [S35]. No backend exists. The local-only half of RxDB is IndexedDB with extra steps
PouchDB plus CouchDB	Local-first sync	Excluded	Requires a CouchDB instance to sync to — the same missing server, plus a database to operate
Yjs	CRDT	Excluded	Solves concurrent multi-writer merge. There is one writer. Also needs a provider or relay server
Automerger	CRDT	Excluded	Same as Yjs. Automerger documents also carry change history, which grows monotonically — a cost with no matching benefit for a single-writer document of the size RP-01 measured

Candidate	Family	Verdict	Reason
ElectricSQL	Local-first sync	Excluded	Requires "a Postgres database and the Electric sync service (a server process)", is read-path only, and writes go back "through your own API" [S34]. Three server-side components this project does not have
localStorage plus versioned JSON export and import	Low-tech baseline	Shortlisted — becomes the fallback and the prerequisite	Zero infrastructure, zero accounts, zero recurring cost, fully portable. Its weakness is that it depends on the user remembering — which is precisely what failed
IndexedDB or OPFS instead of localStorage	Local re-plumbing	Excluded as a durability measure	Larger quota and async writes, but identical exposure to profile deletion. Re-plumbing storage without adding an off-device copy solves nothing this report is about
File System Access API resave to a synced folder	Local plus consumer cloud	Excluded as primary, kept as enhancement	MDN marks <code>showSaveFilePicker()</code> as not Baseline and not working in some of the most widely-used browsers [S5]. Verified present in Chromium. Useful as a one-click "save over yesterday's backup" affordance, never as the load-bearing mechanism
Email the backup to herself	Human process	Excluded	Cannot be automated from a static page without a mail relay, which is a server

3. Master comparison table

Seven candidates against the Q3 criteria, split across three tables to respect the column limit. No criterion is dropped.

3A — Cost, limits and effort

Candidate	Free-tier limits	Cost at this scale	Initial setup effort	Ongoing maintenance
Cloud Firestore, Spark	1 GiB stored, 20K writes/day, 50K reads/day, 20K deletes/day, 10 GiB/month egress, no payment method needed [S6][S7]	Zero, and zero risk of a bill because no card is attached	Create project, choose region, write 4 security-rule lines, add SDK, add sign-in button	SDK version bumps; nothing operational
Supabase Free	2 free projects; Free Plan backups not downloadable [S11][S14]	Zero, but Pro is the documented cure for pausing [S11]	Create project, define table, write RLS policies, add SDK, add auth	Must keep the project awake — see §4

Candidate	Free-tier limits	Cost at this scale	Initial setup effort	Ongoing maintenance
Cloudflare Worker plus KV	100K reads/day, 1,000 writes/day, 1 GB storage, 1 write/sec/key [S16]; Worker 100K requests/day, 10 ms CPU [S17]	Zero	Write, configure and deploy a Worker; design an auth scheme from scratch	Worker runtime and deploy tooling; auth scheme is bespoke code to own forever
Google Drive appDataFolder	Google account storage (15 GiB shared, not re-verified here); whether appData counts against it is TBD	Zero	Cloud project, OAuth client, JavaScript origin, publish the consent screen to escape the 7-day Testing expiry [S26]	Consent-screen configuration drift; Google Identity Services script
Dropbox App folder	Dropbox Basic 2 GB (not re-verified); development status links up to 500 users [S28]	Zero	App console entry, PKCE flow, refresh-token handling in client code	Refresh-token storage and rotation is your code
GitHub Gist or private repo	Unlimited private gists and repos	Zero	Trivial to write, impossible to secure from a static public origin	Token rotation after each inevitable revocation [S30]
localStorage plus JSON export	5 MiB localStorage per origin [S2]; measured quota-bucket 10 GiB unused	Zero	One export button, one import button, one version field	None

3B — What the end user experiences

Candidate	End-user login	Offline behaviour	Multi-device sync	Conflict handling
Cloud Firestore, Spark	One Google sign-in; default web persistence is <code>local</code> , survives browser close, cleared only by explicit sign-out [S10]	Full — local cache is authoritative for the session; SDK queues writes	Yes, automatic, keyed to her Google account	Last-write-wins on a client timestamp; single writer means no real conflicts
Supabase Free	Email or OAuth sign-in	Full if a local cache is kept in front	Yes	Same as above

Candidate	End-user login	Offline behaviour	Multi-device sync	Conflict handling
Cloudflare Worker plus KV	None available without building one; a shared secret is not a login	Full with a local cache	Yes, but anyone who reads the page source can read and overwrite her data	1 write per second per key [S16]; last-write-wins
Google Drive appDataFolder	Google sign-in, then a button press each session to mint a new access token, because the browser token model issues no refresh token [S22]. With prompt set to the empty string the consent screen appears only the first time [S25]	Full with a local cache	Yes	Last-write-wins; Drive supports revision metadata but none is needed
Dropbox App folder	Dropbox sign-in; refresh token can be held client-side so no per-session click [S27]	Full with a local cache	Yes	Last-write-wins
GitHub Gist or private repo	None; the app authenticates as the developer	Full with a local cache	Yes	Last-write-wins, or a merge conflict she cannot resolve
localStorage plus JSON export	None	Total — it is offline by construction	No. She carries a file	None possible; two devices diverge silently

3C — Lock-in, portability, backup, maturity, residency

Candidate	Vendor lock-in	Export and portability	Backup and restore ergonomics	Maturity	EU residency
Cloud Firestore, Spark	Moderate — SDK and query API are proprietary, but the payload is one JSON blob so a migration is a copy	Read the document, write it to a file. Trivial because the schema is one object	Restore is automatic on next sign-in from any device. Keep the JSON export as the escape hatch	High; Google-operated, long-lived	Yes — europe-central2, Warsaw, Poland. Permanent once chosen [S8]

Candidate	Vendor lock-in	Export and portability	Backup and restore ergonomics	Maturity	EU residency
Supabase Free	Moderate; it is Postgres underneath, which helps	pg_dump on paid plans. On Free, backups are not downloadable [S11]	Restore after a pause requires a dashboard click by the maintainer, not the user	High; widely used	Yes — eu-central-1 Frankfurt and five other EU regions [S15]
Cloudflare Worker plus KV	Moderate for KV; the Worker code is yours	GET the key. Simple	Backup is whatever you build	High	Weak. KV is globally replicated; no residency control documented at [S16]
Google Drive appDataFolder	Low on data, high on flow. The file is a plain JSON object in her own Drive	Full — but the folder is hidden from the user and from other apps [S20], so she cannot retrieve it without the app	Restore is automatic once she re-authorises. Deleting the app from My Drive deletes the folder [S20]	High	No control. Consumer Google account storage location is not user-selectable — TBD
Dropbox App folder	Low. A visible folder under Apps in her Dropbox [S28]	Full, and she can see and copy the file herself	Best of the hosted options: the backup is a file she can open	High	TBD . Not researched
GitHub Gist or private repo	Low; it is a text file in git	Full, with version history for free	Excellent for the maintainer, meaningless for her	High	TBD . Not researched
localStorage plus JSON export	None	Total. A JSON file she owns	Depends entirely on her doing it. This is the failure mode that started this programme	n/a	Her own disk, in Poland

The interesting column is 3C's portability row. Drive appDataFolder scores well on lock-in and badly on portability *for the user*, because the folder is invisible to her by design [S20]. If the app were abandoned tomorrow, a Firestore document could be exported by the maintainer, a Dropbox file could be opened by

the teacher, and the `appDataFolder` copy would be unreachable by either without writing code. That asymmetry is a real argument against `appDataFolder` that a feature comparison usually misses, and it is why §6 pairs the primary with a user-visible JSON export rather than treating the cloud copy as sufficient.

4. Free-tier durability and inactivity risk

This is the section that eliminates candidates. The question is not "is the free tier generous" but "does a quiet summer destroy or immobilise the data".

Provider	Documented inactivity policy	Evidence status	Consequence for this app	Repeat-of-incident risk
Supabase Free Plan	"We may pause applications on the Free Plan that exhibit low activity in a 7-day period to save on server resources" [S11]. Free-plan pausing is confirmed by a dedicated docs page: a project is inactive if it "does not receive sufficient user database activity over the past week", and "typically a few user requests to the database each day over the previous week is enough" to avoid it [S12]	Verified, primary source, unambiguous	App breaks after roughly one week of not teaching. Data is not deleted immediately but is unreachable until someone with dashboard access restores it	High — disqualifying
Supabase restore window	Docs say "You can restore a paused project for up to 1 year after it was paused" [S12]. The changelog says "paused Free projects are restorable for 90 days following their pause date", effective 24 June 2024 [S13]	Verified contradiction between two Supabase primary sources	The maintainer cannot tell whether he has 90 days or 365 days to notice. For a hobby project checked rarely, the difference is material	High
Supabase Free backups	"Database backups are not available for download for Free Plan projects" [S11]	Verified	The durable copy cannot be pulled out without upgrading	High
Cloud Firestore, Spark plan	No inactivity pause, suspension or deletion policy is stated on the Firestore usage-and-limits page [S7] or the Firebase pricing page [S6]	Absence of a documented policy, not a guarantee. Recorded as such	No known idle-time failure. Spark also needs no payment method [S6], so there is no card to expire and no bill to go unpaid	Low, with the caveat above

Provider	Documented inactivity policy	Evidence status	Consequence for this app	Repeat-of-incident risk
Cloudflare Workers, KV, D1	No automatic suspension or deletion of inactive Workers is stated at the Workers limits page [S17]; none at the KV limits page [S16]; none at the D1 limits page [S18]	Absence of a documented policy	No known idle-time failure	Low
Neon Free Plan	Compute scale-to-zero "After 5 min" of inactivity and it "cannot be disabled" on Free [S31]. Project-level deletion after prolonged inactivity is not stated on the plans page or the free-plan FAQ	Verified for scale-to-zero. Project deletion TBD	Scale-to-zero is harmless — it resumes. A general dormant-project deletion policy would be fatal and is unresolved	TBD
Appwrite Cloud Free	No inactivity policy of any kind stated on the free-plan page [S32]	Absence of a documented policy on the page that should carry it	Unknown	TBD , and an unknown is a reason to exclude rather than adopt
Google Drive appDataFolder	No project to pause. The folder is deleted when "a user uninstalls your app from their My Drive" or when the user deletes the app's data folder [S20]	Verified	The durability risk moves from the provider to the user's own Drive housekeeping, which is a better place for it	Low on the data. See the token row
Google OAuth, Testing publishing status	"Authorizations by a test user will expire seven days from the time of consent" [S26], and refresh tokens for apps in Testing status expire in 7 days [S19]	Verified, primary source	If the maintainer leaves the consent screen in Testing, the teacher is asked to re-consent every week, forever	High — but avoidable by publishing

Provider	Documented inactivity policy	Evidence status	Consequence for this app	Repeat-of-incident risk
Google OAuth, In production and unverified	Available to any user with a Google Account [S26]. Verification is not required for an app "for your personal use (fewer than 100 users)" [S24]. Unverified published apps do not display the app name and logo on the consent screen, and the unverified-app danger UI applies to sensitive or restricted scopes [S23]. The demo video and security assessment are triggered only by sensitive or restricted scopes [S36]	Verified	<code>drive.appdata</code> is non-sensitive [S21], so no danger UI and no security assessment. The 7-day expiry disappears	Low once published
Dropbox, development status	"It can only be linked with up to 500 total Dropbox users"; the two-week production-approval clock starts "once your app links 50 Dropbox users" [S28]	Verified	One user is nowhere near either threshold. No time-based expiry documented for development status	Low
GitHub	No inactivity policy relevant to gists or repos	Not the failure mode	The failure mode is credential revocation by secret scanning [S30], not inactivity	Medium, from a different cause
<code>localStorage</code> plus JSON export	n/a	n/a	The user is the only durability mechanism	High if it is the <i>only</i> mechanism, low as a second copy

Two things deserve emphasis. First, **Supabase's pause threshold and the teacher's work rhythm are directly incompatible**. A language teacher's schedule has gaps — school holidays, a slow August, a month between course cohorts. Any mechanism that requires database traffic every few days to stay alive imposes a maintenance obligation on the person least able to discharge it. The documented cure is to upgrade to Pro [S11], which violates the zero-to-minimal budget. Keeping the project awake with a cron ping would mean building and operating a keep-alive service for a 14 KB file, which is absurd on its face and is exactly the kind of advice this report is contracted to reject.

Second, **"not documented" is not "does not happen"**. Firestore and Cloudflare are recorded above as having no documented inactivity policy at the pages that would carry one. That is the strongest statement the evidence supports. It is a materially better position than Supabase's documented seven-day pause, and it is not a guarantee. §6's switching triggers assume the policy could change.

5. Fit-to-scale assessment

RP-01 measured the real numbers, so this section does arithmetic rather than speculation.

Dimension	Measured or inferred value	Source
One group, five dates, two months, keys included	354 bytes	RP-01 §3, runtime-measured
Substituted default payment message	446 characters	RP-01 §3, measured
Ten groups over ten teaching months at four lessons each	about 14 KB	RP-01 §3, labelled inference
Entities in the model	3 — Group, MonthlyOverride, plus two global singletons	RP-01 §3
Concurrent writers	1	Contract
Writes per month, order of magnitude	tens — one per group edit, schedule commit or price change	Inferred from RP-01's feature inventory
Reads per month	one per page load	Inferred
Network requests the app makes today	zero after the document itself	RP-01 §6, runtime-verified
Third-party dependencies today	zero	RP-01 §5, runtime-verified

What the RP-01 volume means against each candidate's free tier

Candidate	Relevant limit	This app's share
Firestore stored data	1 GiB [S6]	about 0.0013%
Firestore document size	1 MiB, 1,048,576 bytes [S7]	about 1.4% of one document
Firestore writes	20,000 per day [S6]	one write is 0.005% of a single day's allowance, and a whole month of editing does not reach one day's
Workers KV writes	1,000 per day, 1 per second per key [S16]	ample, but the per-second cap forbids naive write-on-keystroke
Cloudflare D1 max database	500 MB [S18]	about 0.003%
Neon storage	0.5 GB per project [S31]	about 0.003%
localStorage	5 MiB per origin [S2]	about 0.3%

Every candidate is over-provisioned by three to five orders of magnitude. **Capacity is not a decision input.** Anyone choosing between these options on limits is answering the wrong question. The decision inputs are: does the data survive neglect, can she operate it, and can one engineer keep it alive in spare time.

Explicitly rejected as disproportionate

- **Any Postgres.** Supabase, Neon, Vercel Postgres. A relational engine, a schema, migrations, connection pooling and row-level security policies, to hold one JSON object of about 14 KB (RP-01 §3) written by one person. The schema would have one meaningful table and a foreign key to itself. The operational surface exceeds the application.
- **CRDTs — Yjs, Automerge.** Conflict-free replicated data types exist because two writers can edit the same state concurrently and both edits must survive. This app has one writer using one device at a time. The correct conflict policy here is one line: compare `clientUpdatedAt`, keep the larger. Adding a CRDT library adds a dependency, a merge semantics the maintainer must reason about, and monotonically growing change history, in exchange for solving a problem that does not exist. This is the clearest over-engineering trap in the brief's candidate list and it should be named as such.
- **Local-first sync engines — RxDB, PouchDB, ElectricSQL.** All three need a server: RxDB needs `pullHandler`, `pushHandler` and `pullStream` endpoints [S35]; PouchDB needs CouchDB; Electric needs Postgres plus the Electric sync service plus your own write API [S34]. The project has no server and no budget for one. Their offline-first client stories are genuinely good, and irrelevant, because the app is already fully offline.
- **Edge databases — Cloudflare D1.** Relational modelling and SQL migrations for a document that is inherently one object, plus the unsolved question of how a static page proves it is the teacher.
- **A migration to IndexedDB or OPFS presented as a durability fix.** Bigger quota, async API, same death on profile deletion. Worth doing only if volume ever justifies it, which at the volume RP-01 measured it does not.
- **A PWA with a service worker.** Genuinely useful for the WebKit seven-day cap, since home-screen web apps keep their own use counter [S1]. But it is an availability and install-experience feature, not a durability one, and it does not put a byte off the device. Out of scope for this report; a candidate for whichever report covers offline and installability.

What is proportionate

One document. One write on change, debounced. One read on load. One sign-in. One export button. That is the whole design, and §7 describes it.

6. Primary recommendation and fallback, with switching triggers

Primary: one Cloud Firestore document per signed-in user, behind an unchanged local cache

The recommendation. Keep `localStorage` exactly as it is as the session-authoritative cache, and add a single Firestore document, `users/{uid}`, containing the whole application state plus a schema version and a client timestamp. Authenticate with Firebase Auth using Google sign-in only. Pin the database to `europe-central2`, Warsaw [S8]. Stay on the Spark plan, which needs no payment method [S6].

Why it wins, in the order the constraints bite.

1. *Survives neglect.* No documented inactivity pause at the pages that would carry one [S6][S7], and no attached payment method to expire [S6]. Contrast the disqualifying seven-day Supabase pause [S11].
2. *No published secret problem.* Firebase documents that "API keys restricted to Firebase services do not need to be treated as secrets, and it's safe to include them in your code or configuration files", with security enforced by Security Rules and App Check rather than key secrecy [S9]. This is the one candidate family whose published-credential posture is an intended design property rather than a workaround.
3. *Operable by a non-technical teacher.* One Google sign-in. Default web persistence is `local`, which "will be persisted even when the browser window is closed", and "An explicit sign out is needed to clear that state" [S10]. She signs in once and does not think about it again. Compare Drive's browser token model, which issues no refresh token and needs a button press per session [S22].
4. *Maintainer cost near zero.* One project, four lines of security rules, one SDK. No servers, no cron, no dashboard visits, no keep-alive.
5. *EU residency.* `europe-central2` is Warsaw, Poland — the user's own country — and the choice is permanent once provisioned [S8].

The conditions under which this is the correct answer. The teacher has a Google account she can sign into on her own device; the maintainer is willing to own one Google Cloud project and one npm dependency; and the data remains what it is today — lesson dates, prices, group names and a payment template, with no special-category personal data. All three hold.

The honest costs. It adds the app's first third-party dependency to a codebase that RP-01 §5 runtime-verified has exactly zero, and the Firebase JavaScript SDK is materially larger than the entire current 58,649-byte application. It does **not** force a build step — Firebase documents direct ES-module loading from a CDN with per-product imports [S37] — so the zero-toolchain property survives. It puts her data in a project the developer controls, not one she controls. Those costs are worth paying once, and they are the reason the fallback below is not optional.

One asymmetry to state, because it is the same test that demoted Drive. Firebase Auth's persisted session lives in the origin's own storage, and Google's own wording qualifies it with "provided the browser supports this storage mechanism, eg. 3rd party cookies/data are enabled" [S10]. That storage is exactly what WebKit's seven-day script-storage sweep clears [S1][S2]. So on Safari or an iPad she would be signed out roughly weekly and would have to sign in again. This does not flip the verdict, and the reason is worth being precise about: a lost Firebase session costs her one sign-in and her data comes back

automatically, whereas Drive's token model costs a button press **every session on every browser** [S22] and additionally loses its stored token to the same sweep. Firestore's worst case on Safari equals Drive's best case on Chrome. It does, however, promote the browser-and-device question from a footnote to a live input on the primary recommendation — see §8. On Chromium and Firefox, which do not run the seven-day sweep, the session simply persists until explicit sign-out [S10].

Fallback, which ships first: hardened local storage plus a versioned JSON backup

The recommendation. Before any sync exists, make the local store defensible and give her a real backup:

- Wrap `load()` and `save()` at `index.html:1188–1213` in `try / catch`, so a corrupt value produces a visible, recoverable error instead of RP-01's verified dead page.
- Add a **JSON** export and import covering all three keys — `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate` — with a `schemaVersion` field. The existing CSV path is not a backup: RP-01 verified that it omits the template (D16), throws on malformed month keys (D4), and silently destroys existing data on a balanced stray quote (D12).
- Show "Last backup: N days ago" in the toolbar, and nag when N exceeds a threshold.
- Call `navigator.storage.persist()` once on load [S2].

When the fallback is the right answer on its own. If the maintainer cannot or will not run a cloud project; if the teacher's Google account is managed by an employer whose admin policy blocks third-party OAuth apps (Google documents that an admin can set requested services to Restricted, which invalidates tokens [S19]); or if a future report finds the personal-data exposure cannot be cleaned up, making any upload to a third party unacceptable. In those cases local plus disciplined backup is not a hedge, it is the answer — and the visible staleness nag is what makes it materially better than the arrangement that failed.

Switching triggers, and the specific signal for each

Trigger	Specific signal to watch	Action
Firestore free tier acquires an inactivity policy	The Firebase pricing page or the Firestore usage-and-limits page starts documenting pausing, suspension or dormant-project deletion	Fall back to local plus JSON backup; re-run §4
Spark plan starts requiring a payment method	The pricing page stops saying "No payment method needed" [S6]	Fall back. Do not attach a card to a hobby project
Sync is silently broken	The in-app "Last synced" timestamp is older than 14 days while she is actively editing. This indicator is mandatory, not cosmetic — it is the only way a non-technical user can notice a broken write path	Investigate; fall back if unresolved within one teaching week
Her Google account cannot authorise the app	Sign-in fails, or an admin policy blocks it [S19]	Fall back immediately; the JSON export is already there

Trigger	Specific signal to watch	Action
Free-tier write ceiling approached	Firestore document writes exceed 2,000 in a day, i.e. 10% of the 20,000 allowance [S6]	Investigate a write-amplification bug. At the real workload this can only mean a loop
Maintainer capacity ends	No commit in twelve months, or an SDK major version the maintainer will not adopt	Fall back. The local path has no dependencies to rot

The runner-up, and what would select it

Google Drive `appDataFolder` is the second-choice hosted store, and it is close. Its scope is non-sensitive, so there is no verification gauntlet [S21]; there is no provider project to pause; and the bytes sit in the teacher's own Drive. It loses on three verified points: the browser-only token model issues no refresh token and needs a user gesture per session [S22]; the consent screen must be moved out of Testing status or her authorisation expires every seven days [S26][S19]; and the folder is hidden from her, so the "she owns her data" advantage is nominal — she cannot retrieve the file without the app [S20].

Choose Drive over Firestore if the deciding requirement becomes *"the bytes must live in the user's own account, not the developer's project"* — for example if the maintainer wants to stop being a data controller. In that case pair it with the same JSON export so she has a copy she can actually see.

7. Implementation shape for the primary recommendation

RP-01 established that there is no React application today: one HTML file, one inline `<script>`, an App object with `state`, `config`, `handlers`, `render`, `services` and `utils`, and zero build tooling. The shape below is therefore written so it works in the current vanilla file **and** survives the React migration RP-01 §5 anticipates. The sync boundary is deliberately placed where the framework choice cannot reach it.

Document structure

One document per user, at path `users/{uid}`. Not a collection of groups — at 354 bytes measured and about 14 KB projected against a 1 MiB document limit [S7], splitting the state buys nothing and costs atomicity.

```
{
  "schemaVersion": 1,
  "groups": [
    {
      "name": "Beginners A2",
      "price": 200,
      "currency": "PLN",
      "dates": ["2026-08-04", "2026-08-11"],
      "monthlyOverrides": {
        "2026-08": { "price": 77, "dates": ["2026-08-04", "2026-08-11"] }
      }
    }
  ],
}
```

```

"settings": { "defaultCurrency": "PLN" },
"paymentTemplate": "<the user's own template string>",
"clientUpdatedAt": "<epoch milliseconds from Date.now(), stored as a number>",
"serverUpdatedAt": "<Firestore serverTimestamp()>"
}

```

Four notes on this shape. The `groups` array is byte-identical to what `groupLessonPlannerData` holds today (RP-01 §3), so no data transformation is needed in either direction. `paymentTemplate` is carried as a plain string because that is what `index.html:1208` and `1211` store — a raw string, not JSON. `schemaVersion` is the field RP-01 found missing everywhere and is the precondition for any future migration. `clientUpdatedAt` is a plain millisecond number written by the client and is what conflict resolution compares; `serverUpdatedAt` exists only for the maintainer's diagnostics.

Security rules

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}

```

That is the whole authorisation model, and it is what makes the public API key safe [S9]. **Anonymous authentication must not be used.** An anonymous UID is stored in the browser and dies with the profile, which would orphan the document and reproduce the original incident with extra steps. Google sign-in ties the data to an identity that survives the device.

Client library

`firebase-app`, `firebase-auth` and `firebase-firestore` from the modular SDK.

No build step is required. Firebase documents loading the modular SDK directly as ES modules from `https://www.gstatic.com/firebasejs/<version>/firebase-app.js`, with per-product partial imports, for "users with special requirements" who are not using npm [S37]. This matters more than it sounds: it means the primary recommendation can be implemented inside today's single `index.html` with no bundler, no `package.json` and no toolchain — preserving the property RP-01 identified as the app's genuine strength. Firebase does state that npm is the recommended default for most web apps [S37], so the CDN path is a supported alternative rather than the mainline; the version number becomes a hardcoded string in the page that the maintainer must bump by hand.

The remaining cost is honest and singular: this is the project's first third-party dependency, in a tree RP-01 §5 runtime-verified has zero. It is not a toolchain.

Read path on boot

1. Read the three `localStorage` keys and render immediately. The app must be fully usable before any network call resolves — it is today, and that must not regress.

2. Resolve the auth state. If no user is signed in, stop. Local-only operation is a valid steady state, not an error.
3. Fetch `users/{uid}` once.
4. Compare `remote.clientUpdatedAt` with the local value. If remote is greater, replace local state, write it through to `localStorage`, and re-render. If local is greater or remote is absent, push local up. If they are equal, do nothing.
5. Never block rendering on step 3 or 4.

Write path

Every mutation writes `localStorage` synchronously first, exactly as `save()` does today at `index.html:1199–1202`. The remote write is a debounced write-behind — a single `setDoc` with the whole document, coalesced over 1 to 2 seconds of quiet. Whole-document writes are correct here: the payload is about 14 KB (RP-01 §3), and partial updates would require reasoning about which fields changed, which is complexity bought for nothing.

This also fixes a latent bug RP-01 found: `updateDefaultPrice` (`index.html:690–705`) and `handleSelectedDatesPriceChange` (`index.html:858–881`) mutate state without ever calling `save()`. A debounced sync layer that observes state changes rather than being called from individual handlers closes that gap by construction.

Conflict handling

Last-write-wins on `clientUpdatedAt`. One sentence, because there is one writer. The realistic divergence is not concurrent editing but *stale device* — she edits on a laptop, then opens the app on a tablet that was last used in June. Comparing `clientUpdatedAt` before overwriting handles it. Do not add merge semantics for a case that cannot arise; see §5.

Error and offline handling

Condition	Required behaviour
Offline or network error	Silent. Local write already succeeded. Retry on the next mutation or the next load. Never show a modal
Not signed in	The app works fully, local-only. A quiet "Sign in to back up" affordance, not a blocker
<code>permission-denied</code>	Log it and show a persistent but non-blocking banner. This means the rules or the UID are wrong and needs the maintainer
Remote document malformed	Reject it and keep local. Validate <code>schemaVersion</code> and that <code>groups</code> is an array before assigning. RP-01 D14 proves an unguarded assign is a page-killer
<code>QuotaExceededError</code> from <code>localStorage</code>	Catch, warn, do not lose the in-memory state. Unguarded today
<code>JSON.parse</code> failure on load	Catch, keep a copy of the raw bad value under a recovery key, show an actionable message. Never abort <code>init()</code>

Where the sync boundary sits

Outside component state, in a module that owns persistence and knows nothing about the UI.

```
Layer 1  UI                today: App.render.* and App.handlers.*
                             later: React components
                             reads and writes plain state objects only,
                             and never touches storage directly

Layer 2  State             today: App.state
                             later: a store or a context

=== sync boundary ===

Layer 3  services/storage
         loadLocal() and saveLocal()    the three localStorage keys, in try/catch
         exportJson() and importJson()  versioned file backup, no network
         connect() and disconnect()     Firebase Auth
         pull() and pushDebounce()     the single Firestore document
```

The rule that makes this survive the migration: **no component and no render function may call `localStorage` or `Firestore` directly**. They mutate state; the storage module reacts. Today that means `App.services.storage` grows the four new functions and the render layer stops caring. After a React migration the same module is imported by a store; the components never learn that sync exists. This is also the arrangement that makes the fallback in §6 a strict subset — remove `connect`, `pull` and `pushDebounce` and you are left with the local path, still working.

Ordering constraint

Do not implement any of this until the hardcoded personal data at `index.html:387–392` and `index.html:400` is removed. Until then, the default template — which reaches every user who has never edited it — contains a real IBAN and a real tax identifier, and syncing would copy them into a third-party datastore and its backups. Fix that first; §8 records it as a blocker.

8. Risks, contradictions, unknowns and everything marked TBD

Contradiction found between primary sources

Supabase paused-project restore window. The docs page states "You can restore a paused project for up to 1 year after it was paused" [S12]. The changelog, dated 24 June 2024, states "paused Free projects are restorable for 90 days following their pause date" [S13]. Both are Supabase-published. The conditions under which each holds cannot be determined from the pages themselves; the most likely reading is that the changelog is newer and the docs page is stale, but that is inference, not evidence. It does not change the verdict — the seven-day pause trigger [S11] disqualifies Supabase regardless of whether the recovery window is 90 or 365 days — but a maintainer relying on the docs page could be a year wrong about how long he has to notice.

Every TBD in this report

TBD	Why unresolved	What resolves it
Browser and OS of the lost laptop, and of the device she uses now	Not in any input. Determines whether the WebKit seven-day cap [S1] contributed to the loss, and — per §6 — how often the primary recommendation will ask her to sign in again, since Firebase Auth's persisted session lives in the storage that sweep clears [S10] [S2]. This is the single highest-value unknown in the report	Ask the teacher. One question: Safari, Chrome or Edge, on a Mac, a Windows laptop or an iPad
Whether <code>appDataFolder</code> files count against the user's 15 GiB Google storage	Not stated on the appdata guide [S20] or the about-files guide	Check the Drive API <code>about</code> resource <code>storageQuota</code> and <code>quotaBytesUsed</code> documentation, or measure against a test account. Immaterial at 14 KB
Google consumer account data residency	Consumer Google accounts do not expose a region choice	Google Workspace data-regions documentation, which applies to Workspace not consumer accounts. Likely unanswerable in the user's favour
Dropbox short-lived access token lifetime	The OAuth guide says only that tokens "will expire after a short period of time" without a number [S27]. Four hours is widely repeated in Dropbox community threads, which are not sources of record	Inspect the <code>expires_in</code> field returned by the Dropbox <code>/oauth2/token</code> endpoint
Dropbox and GitHub EU data residency	Not researched; both candidates were excluded on other grounds first	Provider trust-centre or DPA documentation
Nhost inactivity policy	Not researched. Excluded because it would not change the decision	Nhost pricing and platform documentation
Neon dormant free-project deletion	Neither the plans page nor the free-plan FAQ states a project-level inactivity policy [S31]. A 90-day deletion figure surfaced in search results but appeared to belong to a region-deprecation announcement, not a standing policy	Read Neon's Azure-regions-deprecation notice and terms of service in full
Appwrite Cloud inactivity policy	Absent from the page that should carry it [S32]	Appwrite terms of service, or ask support
Exact <code>localStorage</code> byte cap in the teacher's browser	The available browser tool blocks Web Storage access, so the probe could not run. MDN documents 5 MiB per origin [S2]	Run an incremental write probe until <code>QuotaExceededError</code> in her browser. Immaterial at 354 measured bytes

TBD	Why unresolved	What resolves it
Which Firebase SDK version string to pin in the CDN import	Resolved as a mechanism, open as a value. Firebase documents CDN ES-module loading with no bundler [S37], so this is a preference, not a blocker. The exact version to pin was not chosen because it will be stale by implementation time	Read the current version from the example on the alt-setup page [S37] at implementation time
Whether her Google account is personal or employer-managed	Not in any input. An administrator can set requested services to Restricted, which invalidates tokens [S19]	Ask the teacher; test the sign-in once

Risks in the recommendation itself

Risk	Severity	Mitigation
Personal financial data replicated to a third party	Critical	Blocker. Remove the hardcoded values at <code>index.html:387-392</code> and <code>index.html:400</code> before writing any sync code. Note that removal from HEAD does not remove them from git history, per RP-01
Stored XSS becomes a credential and data exfiltration path	High	RP-01 D1 verified script execution from a CSV-supplied group name. A signed-in Firestore session in the same page means an injected script can read and rewrite the synced document. Fix D1 before or with the sync work. This risk is strictly worse for candidates that hold a long-lived refresh token in <code>localStorage</code> , which is a further argument against the Dropbox shape
First third-party dependency in a zero-dependency codebase	Medium	Accepted deliberately. The fallback path has no dependencies, so abandonment is survivable
Firestore region choice is permanent [S8]	Medium	Choose <code>europe-central2</code> at creation. There is no second chance
Provider terms change	Medium	The switching triggers in §6 exist for this. The JSON export is the escape hatch and does not depend on the provider
Silent sync failure	Medium	The "Last synced" indicator is mandatory. Without it, a broken write path is invisible until the next data loss
Sync propagates corruption to the cloud copy	Medium	RP-01 D4 showed the app can create data that breaks its own restore path. Validate before pushing, and keep the JSON export as an independent copy that predates any bad write

Where an input was wrong

The research agent contract cites the hardcoded personal data as `index.html:386–392`. Confirmed at first hand by reading the file: line 386 is the heading `Payment details:`; the name is at 387, the IBAN at 388 and the tax identifier at 389. The contract's range also omits the personal first name at line 400 entirely. RP-01 §8 already recorded this correction; this report independently confirms it. Nothing else in either input was found to be wrong — RP-01's §3 and §4 matched the file at every line reference this report checked (`377–378`, `382–400`, `1188–1213`).

9. Sources

#	Title	URL	Accessed	Supports
S1	WebKit — Full Third-Party Cookie Blocking and More	https://webkit.org/blog/10218/full-third-party-cookie-blocking-and-more/	2026-08-20	The seven-day cap on script-writable storage, the exact list of affected storage types, and the home-screen web app exemption
S2	MDN — Storage quotas and eviction criteria	https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria	2026-08-20	Which APIs count toward quota, the 5 MiB Web Storage cap, LRU eviction under pressure, best-effort versus persistent mode, and what <code>navigator.storage.persist()</code> changes
S3	MDN — Window.localStorage	https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage	2026-08-20	localStorage in a private or incognito session is cleared when the last private tab closes
S4	Google Chrome Help — Delete browsing data in Chrome	https://support.google.com/chrome/answer/2392709	2026-08-20	Clearing cookies and other site data deletes Web Storage and Indexed Database data
S5	MDN — Window.showSaveFilePicker()	https://developer.mozilla.org/en-US/docs/Web/API/Window/showSaveFilePicker	2026-08-20	Baseline status read from the rendered page: limited availability, does not work in some of the most widely-used browsers

#	Title	URL	Accessed	Supports
S6	Firebase Pricing	https://firebase.google.com/pricing	2026-08-20	Spark plan needs no payment method; Firestore free allowances of 1 GiB stored, 20K writes, 50K reads and 20K deletes per day, 10 GiB monthly egress; Authentication free tier
S7	Firestore — Usage and limits	https://firebase.google.com/docs/firestore/quotas	2026-08-20	1 MiB maximum document size, 20-level field nesting limit, free-quota figures, and the absence of any inactivity policy
S8	Firestore — Locations	https://firebase.google.com/docs/firestore/locations	2026-08-20	europa-central2 is Warsaw, Poland; the location cannot be changed once provisioned
S9	Firebase — Learn about using and managing API keys	https://firebase.google.com/docs/projects/api-keys	2026-08-20	Firebase API keys are not secrets and are safe in checked-in code; security comes from Security Rules and App Check
S10	Firebase — Authentication state persistence	https://firebase.google.com/docs/auth/web/auth-state-persistence	2026-08-20	Default web persistence is local, survives closing the browser, and is cleared only by explicit sign-out
S11	Supabase — Production checklist	https://supabase.com/docs/guides/platform/going-into-prod	2026-08-20	Free Plan apps with low activity in a seven-day period may be paused; Free Plan backups are not downloadable
S12	Supabase — Project Pausing	https://supabase.com/docs/guides/platform/free-project-pausing	2026-08-20	Definition of an inactive project, the "few requests a day over the previous week" threshold, and the one-year restore claim
S13	Supabase Changelog — Paused Free Plan projects are restorable for 90 days	https://supabase.com/changelog/27497-paused-free-plan-projects-are-restorable-for-90-days	2026-08-20	The contradicting 90-day restore window effective 24 June 2024, and what replaces the restore option afterwards

#	Title	URL	Accessed	Supports
S14	Supabase — About billing on Supabase	https://supabase.com/docs/guides/platform/billing-on-supabase	2026-08-20	Two free projects per user; paused projects do not count toward that limit
S15	Supabase — Regions	https://supabase.com/docs/guides/platform/regions	2026-08-20	EU region identifiers including <code>eu-central-1</code> Frankfurt, used for the residency column
S16	Cloudflare — Workers KV limits	https://developers.cloudflare.com/kv/platform/limits/	2026-08-20	Free plan 100,000 reads and 1,000 writes per day, 1 GB storage, one write per second per key, and no documented inactivity policy
S17	Cloudflare — Workers limits	https://developers.cloudflare.com/workers/platform/limits/	2026-08-20	Free plan 100,000 requests per day, 10 ms CPU, 100 Workers, and no documented suspension of inactive Workers
S18	Cloudflare — D1 limits	https://developers.cloudflare.com/d1/platform/limits/	2026-08-20	Free plan 10 databases, 500 MB maximum database size, 5 GB per account, 50 queries per Worker invocation
S19	Google — Using OAuth 2.0 to Access Google APIs	https://developers.google.com/identity/protocols/oauth2	2026-08-20	Seven-day refresh-token expiry for Testing-status apps, the six-month inactivity expiry, the 100-token-per-client limit, and admin Restricted-service invalidation
S20	Google Drive API — Store application-specific data	https://developers.google.com/drive/api/guides/appdata	2026-08-20	The <code>drive.appdata</code> scope string, that the folder is hidden from the user and other apps, and the two ways it gets deleted
S21	Google Drive API — Choose Google Drive API scopes	https://developers.google.com/workspace/drive/api/guides/api-specific-auth	2026-08-20	<code>drive.appdata</code> and <code>drive.file</code> are classified non-sensitive, while <code>drive</code> and <code>drive.readonly</code> are restricted

#	Title	URL	Accessed	Supports
S22	Google — Using the token model	https://developers.google.com/identity/oauth2/web/guides/use-token-model	2026-08-20	Browser-only clients get no refresh token, and a new access token must be requested from a user-driven event such as a button press
S23	Google — OAuth app state overview	https://developers.google.com/identity/protocols/oauth2/production-readiness/overview	2026-08-20	Unverified published apps do not show name and logo on the consent screen; the danger UI applies to sensitive and restricted scopes
S24	Google Cloud Console Help — When is verification not needed	https://support.google.com/cloud/answer/13464323	2026-08-20	Apps for personal use with fewer than 100 users may continue without verification
S25	Google — Google Identity Services JavaScript API reference	https://developers.google.com/identity/oauth2/web/reference/js-reference	2026-08-20	<code>TokenClientConfig.prompt</code> set to the empty string prompts the user only on the first request
S26	Google Cloud Console Help — Manage App Audience	https://support.google.com/cloud/answer/15549945	2026-08-20	Testing status is limited to 100 test users and authorizations by a test user expire seven days from consent; In production is open to any Google account
S27	Dropbox — OAuth Guide	https://developers.dropbox.com/oauth-guide	2026-08-20	PKCE is supported for public clients including single-page applications; short-lived access tokens with refresh tokens, lifetime unspecified
S28	Dropbox — Developer Guide	https://www.dropbox.com/developers/reference/developer-guide	2026-08-20	Development status links up to 500 users, the two-week production clock starts at 50 linked users, and the App folder permission model
S29	GitHub REST API — Gists	https://docs.github.com/en/rest/gists/gists	2026-08-20	Creating and updating gists requires authentication and the <code>gist</code> scope; anonymous creation is unsupported

#	Title	URL	Accessed	Supports
S30	GitHub Docs — About secret scanning	https://docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning	2026-08-20	Secret scanning runs automatically and free on public repositories, and partner secrets are reported to the provider for revocation
S31	Neon — Neon plans	https://neon.com/docs/introduction/plans	2026-08-20	Free plan 0.5 GB storage and 100 CU-hours per project, and scale-to-zero after five minutes that cannot be disabled
S32	Appwrite — Free plan	https://appwrite.io/docs/advanced/platform/free	2026-08-20	Free plan limits of one organisation, two projects and one database per project, with no inactivity policy stated
S33	PocketBase — Going to production	https://pocketbase.io/docs/going-to-production/	2026-08-20	PocketBase is deployed by uploading the executable to your own server; no managed hosting is offered
S34	Electric — Introduction	https://electric.ax/docs/intro	2026-08-20	Electric requires Postgres plus the Electric sync service, is read-path only, and writes go through your own API
S35	RxDB — Replication	https://rxdb.info/replication.html	2026-08-20	Replication requires a server implementing <code>pullHandler</code> , <code>pushHandler</code> and <code>pullStream</code>
S36	Google Cloud Console Help — OAuth app verification requirements	https://support.google.com/cloud/answer/13464321	2026-08-20	Brand verification requirements, and that only sensitive and restricted scopes trigger the demo video and security assessment
S37	Firebase — Alternative ways to add Firebase to your JavaScript project	https://firebase.google.com/docs/web/alt-setup	2026-08-20	The modular SDK can be loaded directly as ES modules from https://www.gstatic.com/firebasejs/ with per-product partial imports and no bundler; npm remains the recommended default

10. Quick wins

Every item below hardens durability regardless of which candidate in §6 is eventually chosen, so none is blocked by a TBD or an open decision in this report. Deliberately excluded: anything that implements cloud sync (blocked by the personal-data cleanup and by the toolchain TBD), and anything already claimed by RP-01's quick-win list.

Rank	Quick win	Effort	Impact	Basis of ranking
1	Guard the storage layer with try/catch and a recovery path	S	Critical	RP-01 verified that one corrupt value produces a page with no groups, no empty state, an empty month dropdown and dead buttons, unrecoverable through the UI. It is the single most likely mechanism for the next data loss and it is contained in one 26-line service
2	Add a versioned JSON backup covering all three keys	S	Critical	The app's only current backup omits the template, throws on malformed month keys, and can silently destroy data. A whole-state JSON file with a version field is a genuine off-device copy and the prerequisite for every option in §6
3	Show a "Last backup" age and nag when it goes stale	XS	High	The incident's root cause was a missing off-device copy. A visible age turns an invisible omission into a prompt. Pure additive UI, no storage change
4	Request persistent storage on load	XS	Medium	Two lines move the origin out of best-effort eviction and out of WebKit's seven-day script-storage sweep. Verified today that the origin has never asked
5	Confirm before a file import replaces all data	XS	Medium	Import currently replaces everything with no dialog, guarding the app's only restore path. One <code>confirm()</code> on the most destructive path in the app

PROMPT QW-1: Guard the storage layer and add a recovery path

Context: Repo lesson-planner, single deployed file `index.html` at the repo root, served by GitHub Pages. `App.services.storage` at `index.html:1187-1213` has no error handling. `load()` calls `JSON.parse` on the raw `localStorage` value at `index.html:1192` and `index.html:1195` and assigns the result straight to `App.state.groups`. RP-01 runtime-verified that planting a truncated value in `groupLessonPlannerData` throws an uncaught `SyntaxError` that aborts `App.init()` after `cacheElements()` but before the month dropdown is populated at `index.html:413-415`, before `bindEvents()` at 420 and before `render.groups()` at 421, leaving a page with no groups, no empty state, zero month options and inert buttons. `save()` at `index.html:1199-1202` is equally unguarded against `QuotaExceededError` and against the `SecurityError` some browsers raise when storage is blocked.

Task: Wrap each `JSON.parse` in `load()` in its own try/catch so a failure in either key cannot prevent the other from loading and cannot prevent `init()` from completing. On a parse failure, copy the unparseable raw string to a new `localStorage` key by appending the suffix `.corrupt.backup` to the original key name, leave `App.state` at its default, and surface a single non-blocking message telling the user the saved data could not be read and that a copy has been kept. Wrap the two `setItem` calls in `save()` in a try/catch that reports a failure to the user without discarding `App.state`. Add a defensive shape check in `load()` so `App.state.groups` is assigned only when the parsed value is an array, and `App.state.defaultCurrency` only when the parsed settings value is a non-null object.

Constraints: Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape. Do not change the call order inside App.init() at index.html:409–422. Do not introduce a schema version field in this prompt. No new dependencies, no build step. Out of scope: cloud sync, and changing the CSV import or export.

Acceptance criteria: With groupLessonPlannerData set to the string [{"name":"Broken"} and the page reloaded, the month select has 12 options, the group list shows the exact empty-state text No groups yet. Click '+ Add Group' to get started!, clicking + Add Group opens the Add Group modal, and no uncaught error appears in the console. A key named groupLessonPlannerData.corrupt.backup exists and contains the original unparseable string. With groupLessonPlannerSettings set to garbage and groupLessonPlannerData valid, the groups still render. With both keys valid, behaviour and persisted bytes are unchanged from before this prompt.

Verification: Serve index.html locally over http://127.0.0.1, then for each of the three cases above set the localStorage value from the console, reload, and assert the month option count, the group list text content, the presence of the backup key, and an empty console error list. Finally add a group, reload, and confirm the persisted JSON is byte-identical in shape to the RP-01 reference sample.

PROMPT QW-2: Add a versioned JSON backup that covers all three storage keys

Context: Repo lesson-planner, index.html. The only existing backup path is CSV: saveToCsv at index.html:915–931 and loadFromCsv at index.html:935–963. RP-01 verified three defects that make it unfit as a backup – it omits the paymentTemplate key entirely, a date key with a malformed year makes re-import throw Invalid month format and abort the whole restore, and a balanced stray quote is silently accepted and replaces all existing groups with garbage. The app's three localStorage keys are groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, the last being a raw string rather than JSON.

Task: Add a Save Backup button and a Load Backup button to the toolbar at index.html:229–243, alongside the existing Save CSV and Load CSV controls, plus a hidden file input for the backup file mirroring the existing hidden CSV input at index.html:239. Save Backup downloads a single .json file containing an object with exactly four top-level members: schemaVersion set to the number 1, groups holding the parsed contents of groupLessonPlannerData, settings holding the parsed contents of groupLessonPlannerSettings, and paymentTemplate holding the raw string from that key or null when the key is absent. Load Backup reads a chosen .json file, validates that schemaVersion equals 1 and that groups is an array, and only then writes all three localStorage keys and re-renders. On any validation failure it must change nothing and report what was wrong.

Constraints: Do not remove, rename or change the behaviour of the existing CSV export or import. Do not change the three localStorage key names or the persisted data shape of any key; the backup file is a container around the existing shapes, not a new format for them. Reuse the existing Blob and object-URL download approach from index.html:915–931 rather than introducing a library. No new dependencies, no build step. Out of scope: migrating or repairing existing malformed date keys, and any cloud upload.

Acceptance criteria: With two groups, a non-default currency and a customised template present, clicking Save Backup downloads a file whose parsed contents contain schemaVersion equal to 1, a groups array of length 2, a settings object, and the exact template string. Clearing all three keys and then loading that same file restores all three keys so that the group list, the currency formatting and the Edit Template contents all match the pre-export state exactly. Loading a file whose schemaVersion is 2 changes no localStorage key and reports an error. Loading a file whose groups member is an object rather than an array changes no localStorage key and reports an error. Loading a file that is not valid JSON changes no localStorage key and reports an error. The CSV export and import continue to behave exactly as before.

Verification: Serve locally, build the two-group fixture, click Save Backup and inspect the downloaded JSON. Clear localStorage, reload, click Load Backup with that file, and assert the three keys byte-for-byte against values captured before the export. Then hand-edit copies of

the file for each of the three rejection cases and confirm after each attempt that `Object.keys(localStorage)` is unchanged.

PROMPT QW-3: Show how long ago the last backup was taken and nag when it is stale

Context: Repo lesson-planner, `index.html`. The app gives the user no signal about whether an off-device copy of her data exists. RP-01 records that all stored data was previously lost when the user's laptop was cleaned, and that the app's only persistence is three `localStorage` keys. There is currently no record anywhere of when a backup was last taken. This prompt depends on a backup action existing; it must therefore wire itself to whichever download action is present in `index.html` at the time it is implemented, which is Save CSV at `index.html:915-931` today and additionally Save Backup if the JSON backup prompt has already landed.

Task: On every successful backup download, write the current time as an ISO 8601 string to a new `localStorage` key named `lastBackupAt`. On page load, render a short status line in the toolbar area at `index.html:229-243` reading either Never backed up when the key is absent, or Last backup: today, or Last backup: N days ago. When the key is absent or the age exceeds 14 days, give that status line a visually distinct warning treatment using the existing danger colour already present in the stylesheet.

Constraints: Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape of any of them; `lastBackupAt` is a fourth, additive key that no other code reads. Do not add the new key to the `clear()` function at `index.html:1203-1206` in this prompt. Do not block, modal or interrupt the user; this is a passive indicator only. Ensure the status line is announced to assistive technology by giving it an appropriate live-region attribute. No new dependencies, no build step.

Acceptance criteria: On a profile with no `lastBackupAt` key, the status line reads exactly Never backed up and carries the warning treatment. After clicking a backup download, the key holds a parseable ISO 8601 timestamp and the line reads exactly Last backup: today without the warning treatment. Setting the key to a timestamp 3 days in the past yields Last backup: 3 days ago without the warning treatment. Setting it 15 days in the past yields Last backup: 15 days ago with the warning treatment. No dialog or alert is shown at any point. The three existing keys are untouched by all of the above.

Verification: Serve locally, and for each of the four states above set or clear `lastBackupAt` from the console, reload, and assert the rendered text content of the status element and the presence or absence of its warning class. Trigger a real backup download once and assert the key was written and is parseable by `Date`. Confirm `Object.keys(localStorage)` gains only `lastBackupAt`.

PROMPT QW-4: Ask the browser for persistent storage on load

Context: Repo lesson-planner, `index.html`. The app stores everything in `localStorage` and has never requested persistent storage. Measured on the live origin <https://denyslystopadskyy.github.io> in Chromium on 2026-08-20: `navigator.storage.persisted()` returned false and `navigator.storage.persist` is available. Browsers evict best-effort origins under storage pressure and WebKit additionally clears an origin's script-writable storage after seven days of browser use without user interaction on the site; origins granted persistence are exempt from both.

Task: In `App.init()` at `index.html:409-422`, after the storage load call at `index.html:411`, add a guarded call that requests persistent storage when the API is available and persistence has not already been granted. The call must be fully feature-detected, must not await anything on the critical rendering path, and must never throw into `init()`. Record the outcome on the existing `App.state` object at `index.html:363-373` as a new property named `storagePersisted`, initialised to null and set to the boolean result once known, so a test can read it back via `page.evaluate`. Do not surface it in the UI.

Constraints: Must not block or delay rendering, and must not change the order or the synchronous behaviour of any existing statement in `App.init()`. Must be a no-op with no

console error when `navigator.storage` or `navigator.storage.persist` is absent, leaving `App.state.storagePersisted` as null. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Do not add a permission prompt of your own and do not react to a denial with a dialog; a denial is an acceptable outcome. No new dependencies, no build step. Acceptance criteria: On a fresh load over HTTPS or `http://127.0.0.1` in a Chromium browser, `App.state.storagePersisted` settles to either true or false within one second of load, and the console error list is empty. When `navigator.storage.persisted()` already returns true before load, `navigator.storage.persist` is never invoked. When `navigator.storage` is undefined, the page still loads, the group list and month dropdown render, + Add Group opens the modal, `App.state.storagePersisted` remains null, and the console error list is empty. The three existing `localStorage` keys and their contents are byte-identical before and after the change for the same user actions.

Verification: Serve locally over `http://127.0.0.1` and load the page; read `App.state.storagePersisted` via `page.evaluate` and assert it is a boolean, and assert the console error list is empty. Wrap `navigator.storage.persist` in a counting spy installed before load and assert it was called zero times when `navigator.storage.persisted()` already resolves true. Then delete `navigator.storage` from the window before load and assert `App.state.storagePersisted` is null, the group list renders, and the console error list is empty.

PROMPT QW-5: Confirm before a file import replaces all existing data

Context: Repo `lesson-planner`, `index.html`. `loadFromCsv` at `index.html:935-963` replaces `App.state.groups` wholesale and persists at `index.html:948` with no confirmation of any kind. RP-01 runtime-verified that importing a file silently replaced two existing groups with the imported content and showed zero dialogs, and separately that a CSV containing a balanced stray quote replaced both real groups with a single garbage group, again with no dialog. By contrast Clear All Data at `index.html:964-975` does ask, using the text Clear all groups and schedules? This cannot be undone. Import is the app's only restore path, so an accidental import is a data-loss event on the very control the user reaches for during a recovery.

Task: Before any imported file is applied, and only when `App.state.groups` is non-empty, show a confirm dialog naming the number of existing groups that will be replaced and requiring explicit approval. On cancellation, apply nothing, leave all `localStorage` keys untouched, and reset the file input so the same file can be chosen again afterwards. Apply the same guard to every file-import entry point present in `index.html`, which is the CSV import today and the JSON backup import as well if that prompt has already landed.

Constraints: Do not change the parsing, validation or error messages of the existing importers, and keep the existing alert wording Unable to load CSV: followed by the underlying message intact. Do not add a confirmation when there are zero existing groups, since nothing can be lost. Do not change the three `localStorage` key names `groupLessonPlannerData`, `groupLessonPlannerSettings` and `paymentTemplate`, and do not change the persisted data shape. Use the same confirm mechanism already used by Clear All Data rather than introducing a custom modal. No new dependencies, no build step.

Acceptance criteria: With two groups present, choosing an import file shows a confirm dialog whose text includes the number 2. Cancelling leaves the group list, the month rows and all `localStorage` keys exactly as they were, and choosing the same file again still shows the dialog. Accepting performs the import exactly as before this change. With zero groups present, choosing an import file shows no dialog and imports directly. A malformed file that previously threw still produces the same error message, and only after the confirmation has been accepted.

Verification: Serve locally, create two groups, export a backup or CSV, then re-import it and cancel; assert the two original group names and every `localStorage` value are unchanged, then re-import and accept and assert the import applied. Repeat the cancel case twice in a row with the same file to prove the input was reset. Then clear all data and confirm an import proceeds with no dialog.